Branch/Class/Batch: AIDS TE A1
Student Roll no. 13
Student Name: SAKSHI RANE

Assignment 1:

Network Topology and Media Simulation

1. Client-Server Socket Programming

Aim

To implement a simple client-server network architecture using Python socket programming, where a client sends a text message to a server and receives a response.

Objective

The objective of this experiment is to understand the basic concept of client-server communication using socket programming. The client establishes a connection with the server, sends a text message, and receives an acknowledgement from the server.

Theory

Socket Programming

Socket programming is a method of communication between two computers or processes over a network. A socket acts as an endpoint for sending and receiving data.

In this experiment, a TCP socket is used to establish reliable communication between a client and a server.

Client:
The client initiates the connection with the server and sends a message.

Server:
The server waits for incoming connections, receives the client's message, and sends a response.

IP Address:
The program uses 127.0.0.1, which is the loopback address representing the local computer.

Port Number:
Port 5000 is used for communication between the client and server.

TCP:
TCP (Transmission Control Protocol) provides reliable, connection-oriented communication.

Algorithm

Server Algorithm

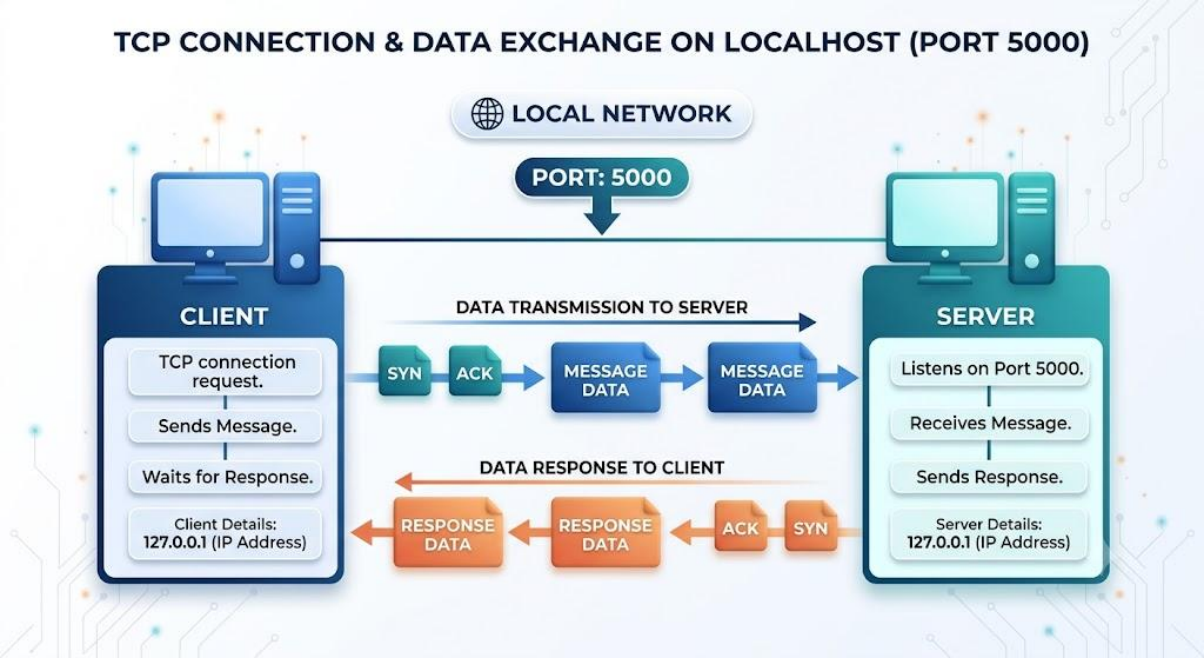
1. Import the socket module.
2. Create a TCP socket.
3. Assign the IP address 127.0.0.1 and port 5000.
4. Bind the socket to the specified IP address and port.
5. Start listening for client connections.
6. Accept the client connection.

7. Receive the message sent by the client.
8. Display the received message.
9. Send a response to the client.
10. Close the client and server sockets.

Client Algorithm

1. Import the socket module.
2. Create a TCP socket.
3. Specify the server IP address and port.
4. Connect to the server.
5. Accept a message from the user.
6. Send the message to the server.
7. Receive the server's response.
8. Display the response.
9. Close the client socket.

Network Architecture



SOURCE CODE

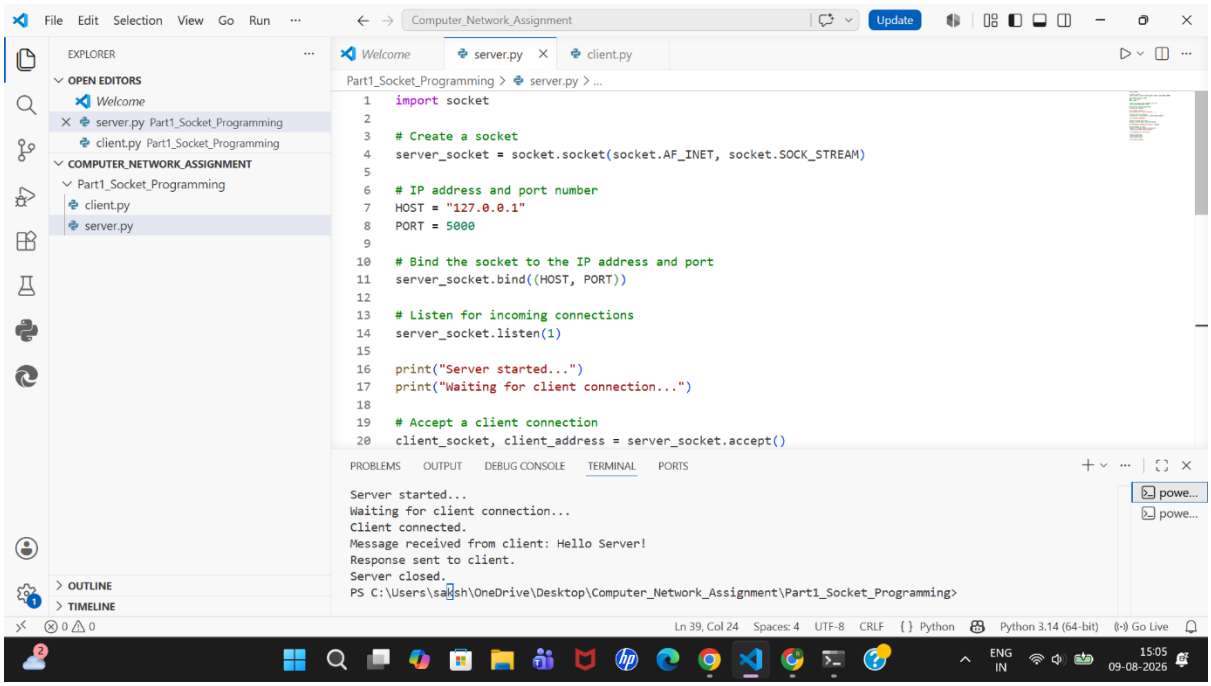
Server Program

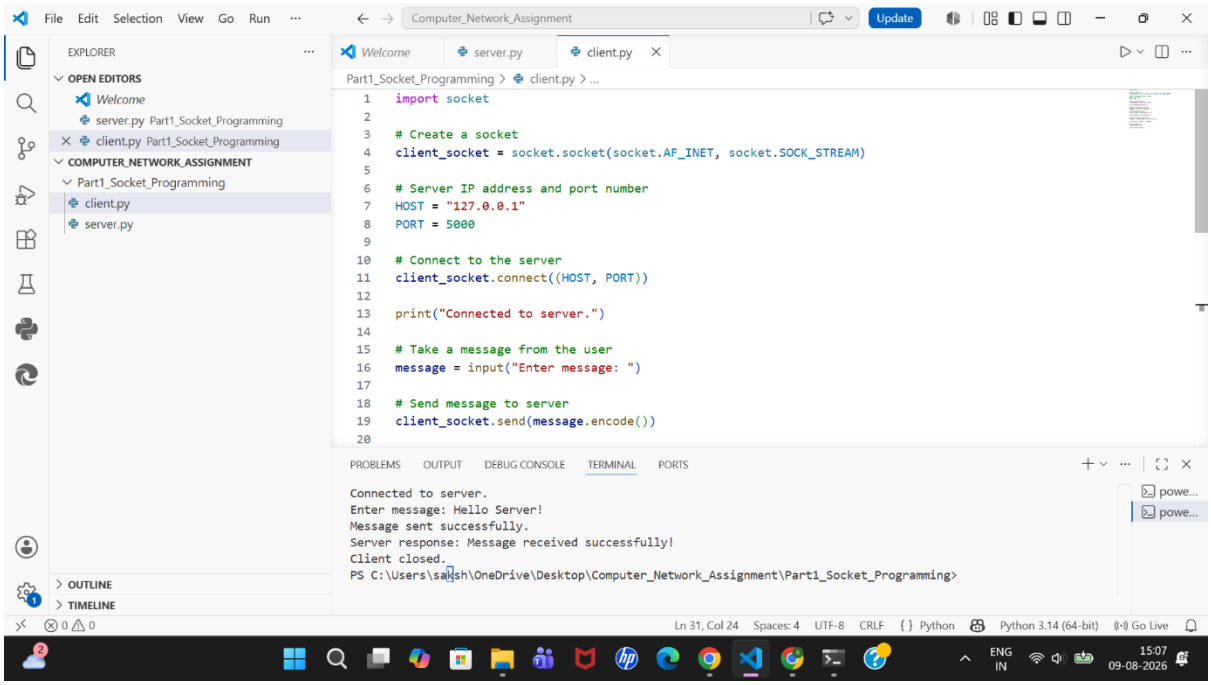
```python
import socket

# Create a socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# IP address and port number
HOST = "127.0.0.1"
PORT = 5000

# Bind the socket to the IP address and port
server_socket.bind((HOST, PORT))

# Listen for incoming connections
server_socket.listen(1)

print("Server started...")
print("Waiting for client connection...")

# Accept a client connection
client_socket, client_address = server_socket.accept()

print("Client connected.")

# Receive message from client
message = client_socket.recv(1024).decode()

print("Message received from client:", message)

# Send response to client
response = "Message received successfully!"
client_socket.send(response.encode())

print("Response sent to client.")

# Close connections
client_socket.close()
server_socket.close()

print("Server closed.")
```

Client Program

```python
import socket

# Create a socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Server IP address and port number
HOST = "127.0.0.1"
PORT = 5000

# Connect to the server
client_socket.connect((HOST, PORT))

print("Connected to server.")

# Take a message from the user
message = input("Enter message: ")

# Send message to server
client_socket.send(message.encode())

print("Message sent successfully.")

# Receive response from server
response = client_socket.recv(1024).decode()

print("Server response:", response)

# Close connection
client_socket.close()


print("Client closed.")
```

OUTPUT





Result

The client-server architecture was successfully implemented using Python socket programming. The client established a TCP connection with the server using the loopback IP address 127.0.0.1 and port 5000. The client successfully sent a text message to the server, and the server successfully received the message and returned a response.

2. Network Device and Transmission Media Classification

Aim

To develop a Python program that classifies common network devices and transmission media based on their OSI layer, type, and primary function.

Objective

The objective of this experiment is to understand the role of common networking devices and transmission media in a network topology and classify them according to their layer of operation and primary function.

Theory

Switch

A switch is a networking device primarily operating at **Layer 2 – Data Link Layer** of the OSI model. It connects multiple devices within a LAN and forwards frames using MAC addresses.

Router

A router operates primarily at **Layer 3 – Network Layer**. It connects different networks and forwards packets based on IP addresses.

Bridge

A bridge operates at **Layer 2 – Data Link Layer**. It connects and filters traffic between two LAN segments.

Access Point

An access point primarily operates at **Layer 2 – Data Link Layer** and provides wireless connectivity between Wi-Fi devices and a wired network.

Transmission Media

Twisted Pair Cable

Twisted pair is a wired transmission medium commonly used for Ethernet and LAN connections.

Coaxial Cable

Coaxial cable is a wired medium used for cable television, broadband and data transmission.

Fiber Optic Cable

Fiber optic cable transmits data using light and provides high-speed communication over long distances.

Wireless/Radio

Wireless transmission uses radio waves for communication. Wi-Fi is a common example.

Classification Table

Device/Media	Layer/Type	Primary Function
Switch	Layer 2 – Data Link	Connects devices within a LAN
Router	Layer 3 – Network	Connects different networks
Bridge	Layer 2 – Data Link	Connects LAN segments
Access Point	Layer 2 – Data Link	Provides wireless connectivity
Twisted Pair	Wired	Ethernet/LAN connections
Coaxial Cable	Wired	Broadband/cable transmission
Fiber Optic	Wired	High-speed long-distance transmission
Wireless/Radio	Wireless	Wi-Fi and wireless communication

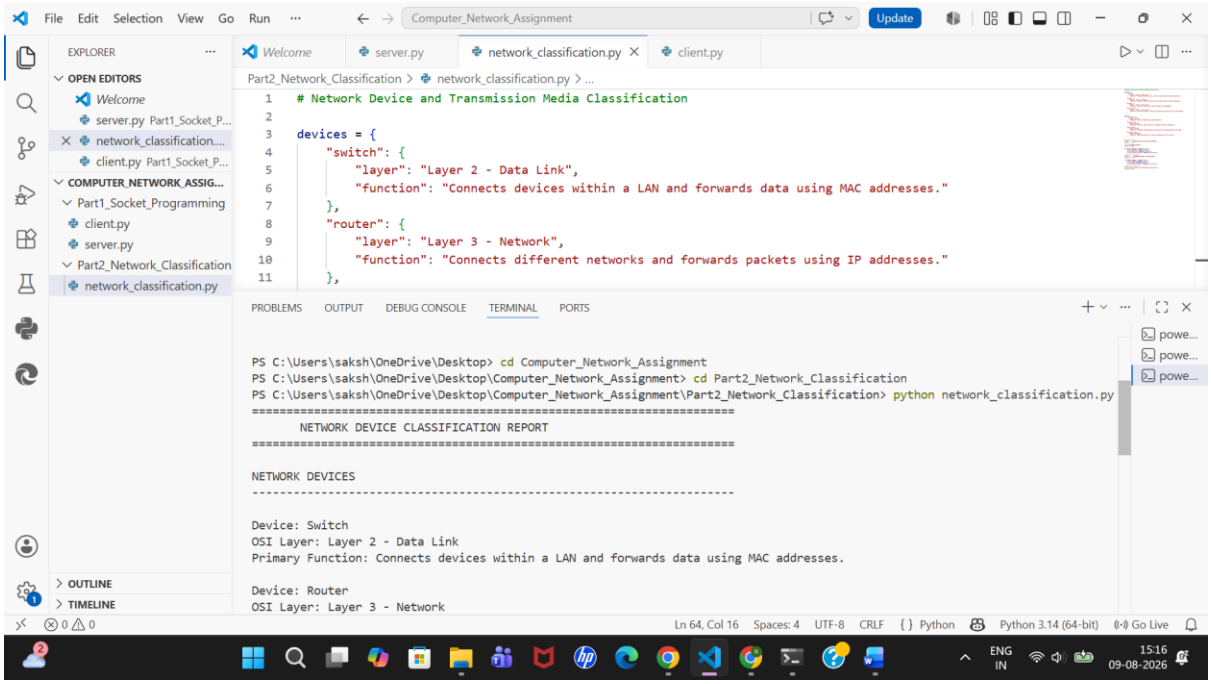
SOURCE CODE

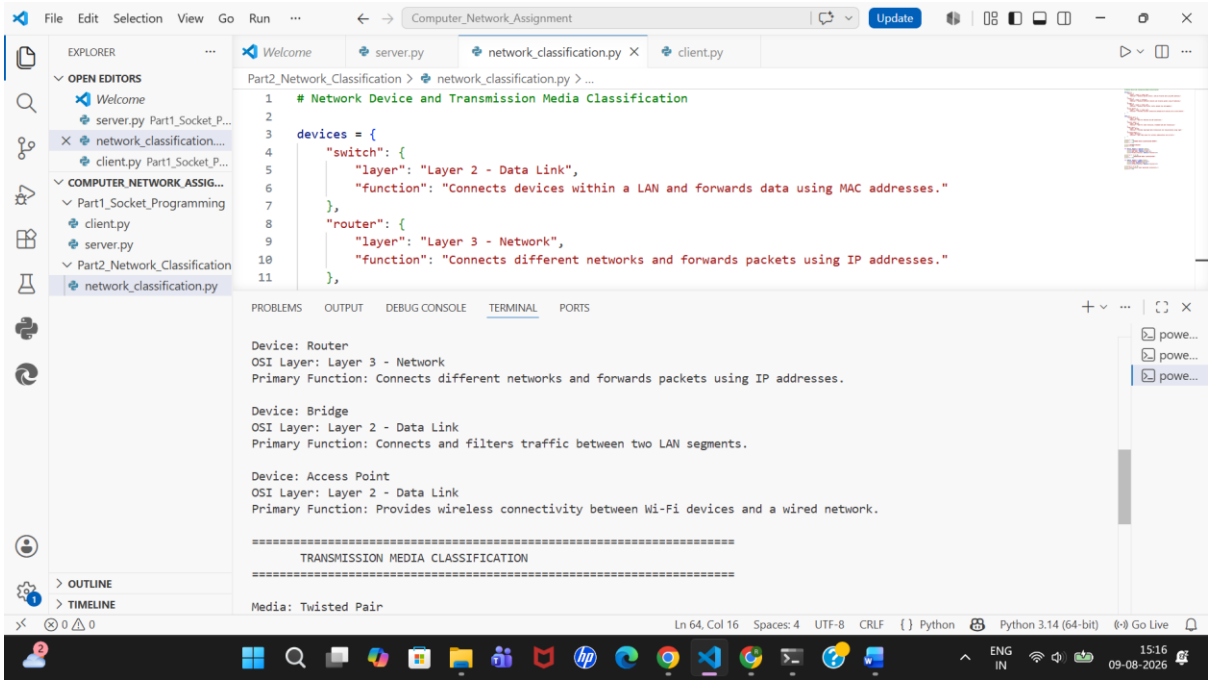
```
# Network Device and Transmission Media Classification

devices = {
    "switch": {
        "layer": "Layer 2 - Data Link",
        "function": "Connects devices within a LAN and forwards data using MAC addresses."
    },
    "router": {
        "layer": "Layer 3 - Network",
        "function": "Connects different networks and forwards packets using IP addresses."
    },
    "bridge": {
        "layer": "Layer 2 - Data Link",
        "function": "Connects and filters traffic between two LAN segments."
    },
    "access point": {
        "layer": "Layer 2 - Data Link",
        "function": "Provides wireless connectivity between Wi-Fi devices and a wired network."
    }
}

media = {
    "twisted pair": {
        "type": "Wired",
        "function": "Used for Ethernet and LAN connections."
    },
    "coaxial cable": {
        "type": "Wired",
        "function": "Used for cable television, broadband and data transmission."
```

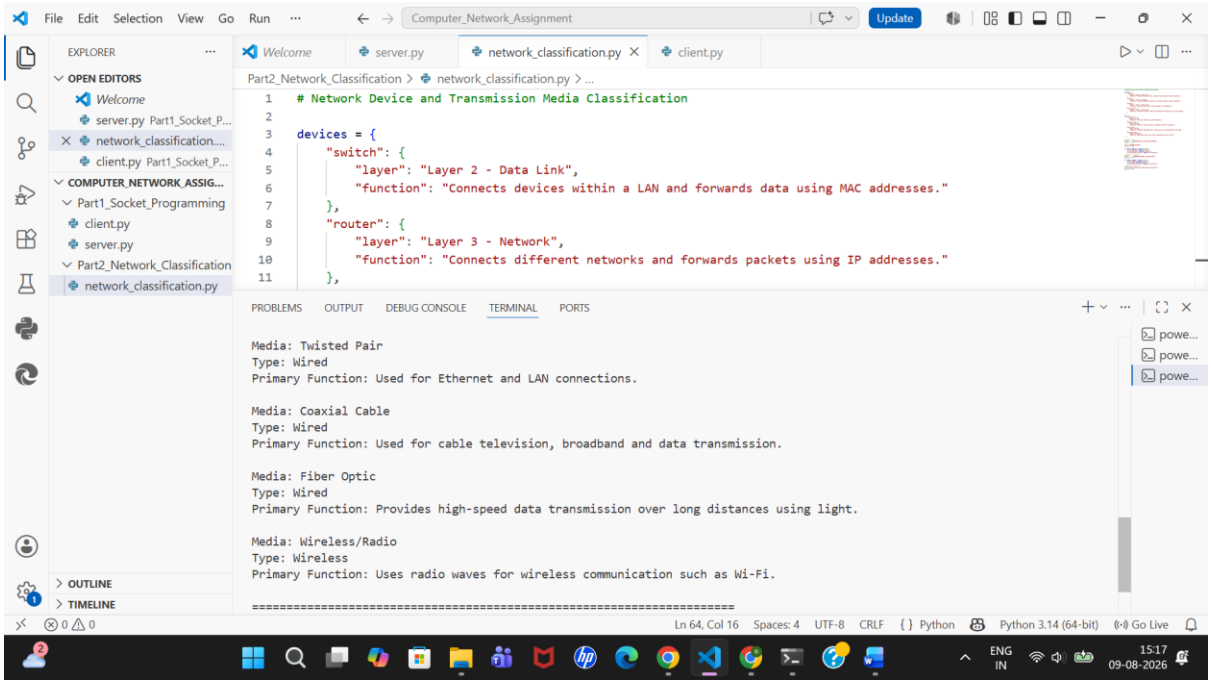
```
    },
    "fiber optic": {
        "type": "Wired",
        "function": "Provides high-speed data transmission over long distances using light."
    },
    "wireless/radio": {
        "type": "Wireless",
        "function": "Uses radio waves for wireless communication such as Wi-Fi."
    }
}

print("=" * 70)
print("       NETWORK DEVICE CLASSIFICATION REPORT")
print("=" * 70)

print("\nNETWORK DEVICES")
print("-" * 70)

for device, details in devices.items():
    print(f"\nDevice: {device.title()}")
    print(f"OSI Layer: {details['layer']}")
    print(f"Primary Function: {details['function']}")

print("\n" + "=" * 70)
print("       TRANSMISSION MEDIA CLASSIFICATION")
print("=" * 70)

for medium, details in media.items():
    print(f"\nMedia: {medium.title()}")
```

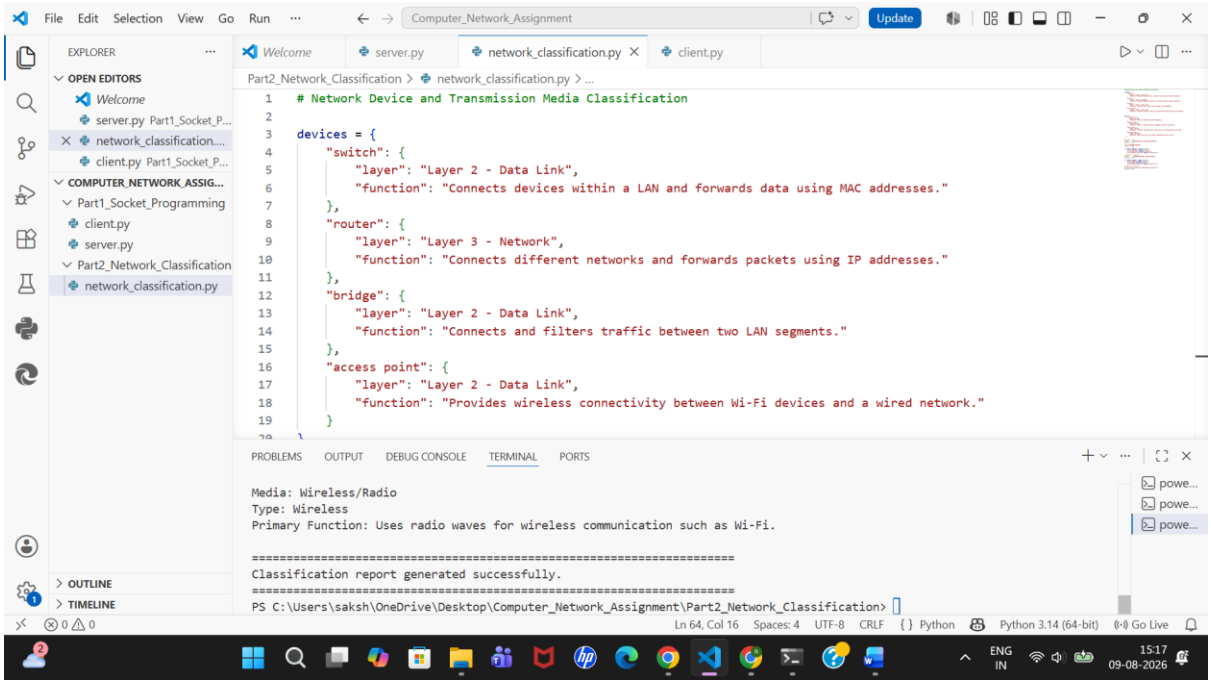
```
    print(f"Type: {details['type']}")
    print(f"Primary Function: {details['function']}")

print("\n" + "=" * 70)
print("Classification report generated successfully.")
print("=" * 70)
```

Output




File Edit Selection View Go Run
Computer_Network_Assignment
Update
EXPLORER
OPEN EDITORS
Welcome
server.py Part1_Socket_P...
network_classification....
client.py Part1_Socket_P...
COMPUTER_NETWORK_ASSIG...
Part1_Socket_Programming
client.py
server.py
Part2_Network_Classification
network_classification.py
OUTLINE
TIMELINE
Welcome
server.py
network_classification.py
client.py
Part2_Network_Classification > network_classification.py > ...
1 # Network Device and Transmission Media Classification
2
3 devices = {
4 "switch": {
5 "layer": "Layer 2 - Data Link",
6 "function": "Connects devices within a LAN and forwards data using MAC addresses."
7 },
8 "router": {
9 "layer": "Layer 3 - Network",
10 "function": "Connects different networks and forwards packets using IP addresses."
11 },
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Device: Router
OSI Layer: Layer 3 - Network
Primary Function: Connects different networks and forwards packets using IP addresses.
Device: Bridge
OSI Layer: Layer 2 - Data Link
Primary Function: Connects and filters traffic between two LAN segments.
Device: Access Point
OSI Layer: Layer 2 - Data Link
Primary Function: Provides wireless connectivity between Wi-Fi devices and a wired network.
TRANSMISSION MEDIA CLASSIFICATION
Media: Twisted Pair
Ln 64, Col 16 Spaces: 4 UTF-8 CRLF { } Python Python 3.14 (64-bit) Go Live
ENG IN
15:16
09-08-2026


File Edit Selection View Go Run
Computer_Network_Assignment
Update
EXPLORER
OPEN EDITORS
Welcome
server.py Part1_Socket_P...
network_classification....
client.py Part1_Socket_P...
COMPUTER_NETWORK_ASSIG...
Part1_Socket_Programming
client.py
server.py
Part2_Network_Classification
network_classification.py
OUTLINE
TIMELINE
Welcome
server.py
network_classification.py
client.py
Part2_Network_Classification > network_classification.py > ...
1 # Network Device and Transmission Media Classification
2
3 devices = {
4 "switch": {
5 "layer": "Layer 2 - Data Link",
6 "function": "Connects devices within a LAN and forwards data using MAC addresses."
7 },
8 "router": {
9 "layer": "Layer 3 - Network",
10 "function": "Connects different networks and forwards packets using IP addresses."
11 },
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Media: Twisted Pair
Type: Wired
Primary Function: Used for Ethernet and LAN connections.
Media: Coaxial Cable
Type: Wired
Primary Function: Used for cable television, broadband and data transmission.
Media: Fiber Optic
Type: Wired
Primary Function: Provides high-speed data transmission over long distances using light.
Media: Wireless/Radio
Type: Wireless
Primary Function: Uses radio waves for wireless communication such as Wi-Fi.
Ln 64, Col 16 Spaces: 4 UTF-8 CRLF { } Python Python 3.14 (64-bit) Go Live
ENG IN
15:17
09-08-2026



Result

The Python program successfully classified the given network devices according to their OSI layer and primary function. It also classified different transmission media based on their type and application.

Conclusion

Through this assignment, the basic concepts of computer networking and network communication were studied and implemented. A client-server architecture was successfully created using Python TCP socket programming. The communication between the client and server demonstrated the process of establishing a connection, sending data, receiving data, and terminating the connection. Additionally, common network devices and transmission media were classified according to their network layer, type, and primary function.